# Draft A

# Hydrofrac

*Lineage-driven fault injection for Hydro: pump in faults until a labelled collection cracks.*

Hydrofrac checks whether a Hydro program's `EventualConsistency` labels survive the faults its own transports admit. It runs the program in the Hydro simulator, records why each run produced its outputs, and uses that record to decide which faults to try next. The search ends in one of two ways:

- A **counterexample**: a set of faults, within declared bounds, under which the surviving members of a cluster disagree about a labelled collection. Hydrofrac prints the faults and the schedule.
- A **certificate**: no fault set within those bounds caused disagreement in any run it explored.

The programmer does not write a specification. If a cluster collection is labelled `EventualConsistency`, the label is the specification.

The development record, with the reasoning behind each decision, is in `history-hydrofrac-development.md`.

## Why hydrofrac is needed

A collection at a Hydro cluster carries a consistency level in its location type, `Cluster<'a, Tag, Con>`. The default, `NoConsistency`, makes no promise. `EventualConsistency` promises that every member of the cluster that is given the same inputs ends up holding the same contents.

A collection gets the `EventualConsistency` label in one of three ways:

1. **A transport promises it.** For example, `broadcast_closed` over a transport whose `ConsistencyGuarantee` is `EventualConsistency`.
2. **A programmer asserts it.** They call `assert_has_consistency_of(manual_proof!(...))`.
3. **An operator preserves it.** Operators whose algebra carries the label from input to output do this when applied to a labelled input. Examples are a `fold` with a commutative, idempotent combiner, a `map`, or a `filter`.

Collections at a single `Process` carry no label, because there is only one holder.

The type system checks the label conditionally. It verifies that the operators between the inputs and the collection are deterministic enough that members with the same inputs compute the same contents. It cannot check the condition itself: that every member actually receives the same inputs. That condition is false in exactly the executions we worry about in distributed systems:

- A sender crashed partway through a broadcast, so some members got the message and others did not.
- A message on a lossy channel was dropped and never resent.

In those executions the label promises nothing, even though the type system accepted it.

The transports grant the label on informal grounds. The comments in `networking/mod.rs` reason about the recipient ("a failed connection models the recipient as failed") or about delay ("dropped messages are

indefinitely delayed"). Neither argument holds once the sender crashes or a message is lost for good. Whether the label is true is a property of the program built on the transport, and specifically of whether that program retransmits or re-disseminates. The type system cannot see that. Hydrofrac can.

In this document, **complete EC** means the label's conclusion together with its premise. The type system supplies the conclusion: same inputs give the same contents. Hydrofrac checks the premise: under the admitted faults, every surviving member gets every input that any other survivor got. A collection that passes both has complete EC within hydrofrac's bounds.

## A reminder about LDFI

Hydrofrac is Molly (Alvaro, Rosen and Hellerstein, SIGMOD 2015) with the Hydro simulator in place of Dedalus.

The central idea of lineage-driven fault injection is to work backwards from a good outcome instead of forwards from guessed faults. A random fault injector picks faults and hopes one of them matters. LDFI does the following instead:

1. Run the program once with no faults and observe a correct outcome.
2. Ask why the outcome happened. Lineage gives the answer as the set of message deliveries the outcome depended on. Any fault set that breaks the outcome must kill at least one of those deliveries. Any other fault set leaves the support for the outcome intact.
3. Choose a fault set that kills at least one delivery from each "why" collected so far. Formally, this is a hitting set of the derivations.
4. Run the program under that fault set. There are two possible results: - The outcome breaks. That is a counterexample. - The outcome survives. The program found another way to produce it, through a retransmission, a redundant path, or an echo. Its lineage is a new "why", and the next fault set must hit that one as well.
5. Repeat until either a fault set breaks the outcome, or no fault set within the bounds can hit every "why". The second case is a certificate.

Lineage keeps the search small. Faults that cannot affect the outcome are never tried, and each surviving run shows the search exactly which redundancy it must defeat next.

Molly's users wrote the correctness property by hand. For eventual consistency, hydrofrac takes the property from the label.

## Using hydrofrac

Nothing is required of the program under test. The test does three things:

- Fixes one concrete execution: the topology, the cluster sizes, and the inputs.
- Names the labelled collection to check.
- Drives the run.

```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<u32, TotalOrder, ExactlyOnce>();
```

```
let delivered = reliable_broadcast_closed(items, &members);   // labelled EventualConsistency

let mut spec = Spec::new(&mut flow);
let correct = spec.correct_members(&members, 0..3);
let observed = spec.observe_cluster(delivered.weaken_consistency());
spec.eventual_consistency(observed, correct);

let compiled = flow.sim()
    .skip_consistency_assertions()      // the simulator does not yet check them
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

Ldfi::new()
    .level(Level::Record)
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search(&compiled, async |_run| {
        send.send(1);
        send.send(2);
        hydro_lang::sim::quiesce().await;
    })
```

The test body defines the execution: which inputs arrive, which timers fire, and how long the run lasts. Some protocols need time after the faults stop in order to converge, for example those that use retransmission timers, gossip rounds, or leader change. For those, the body should fire the relevant timers after `run.end_of_finite_failures()`.

The result prints as one of the following:

- A counterexample, with its faults and the failing schedule.
- A certificate, with its grade, the number of hypotheses tried, and the exploration mode.
- `Vacuous`: the failure-free run delivered nothing, so there was nothing to check.
- `Budget`: the hypothesis budget ran out before the search finished.

Two alternative strategies use the same fault universe and serve as ablations that measure what lineage contributes:

- `Strategy::Blind` runs the same loop without lineage. It tries untried fault sets in order of increasing size.
- `Ldfi::fuzz_faults_with` samples fault sets at random.

## How hydrofrac works

### The probe run and the specification

Hydrofrac first runs the program with no faults. This is the **probe**. It records every element that ends up in the labelled collection `C` at any member. That set is `FACTS`: what the program delivers when nothing goes wrong.

In every later run, which includes faults, hydrofrac ignores the members that crashed and checks one condition:

> No element of `FACTS` is held by one surviving member and missing from another.

Survivors must agree. They do not need to hold all of `FACTS`. Some faults keep an element out of the system entirely, for example when its only sender dies before sending it to anyone. Then no survivor has the element,

and that is not a disagreement.

Two edge cases:

- If `FACTS` is empty, there is nothing to check, and the search reports `Vacuous`.
- If a fault set leaves the survivors holding nothing at all, the condition holds trivially. That run counts neither for nor against the program.

The specification is itself a Hydro flow, `Spec::eventual_consistency`. It runs at a dedicated observer process that is compiled into the program:

1. `spec.observe_cluster` connects the labelled collection to the observer.
2. `spec.correct_members` names the members of the cluster.
3. Before each run, hydrofrac tells the observer which members the current hypothesis does not crash, and gives it `FACTS` from the probe.
4. The observer computes the distinct `(member, fact)` pairs over `FACTS` at correct members.
5. At the end of the run, the specification holds if that count equals the number of facts present times the number of correct members.

The observer is part of the compiled flow. Its output therefore has lineage like any other output, and the same machinery that explains program outputs also explains why the specification held.

## Fault points

Every `Network` operator in the program is a fault point, and the transport it names determines which faults it admits. Programs do not register fault points. The transports they already declare are the fault model. Hydrofrac places a filter in front of every network send. The filter can do two things:

- **Omit a message.** Omission is available only on lossy channels such as `lossy_delayed_forever`. A message is identified by channel, sender, destination, and wire bytes. While failures are active, every byte-identical copy on that channel is dropped, so a retransmission of the same payload counts as the same message.
- **Crash a node.** Crashes are available only on nodes the test registers as crashable, and each one counts against the crash budget. The fault `crash(node after m)` means the node performs every send up to and including message `m`, and every later send is dropped. The node's external outputs are also suppressed.
- The crash point is named by a message, not by a send count, so the fault means the same thing in every schedule.
- Naming it by message also lets a crash fall between two sends of a single broadcast. This is the case that breaks naive dissemination.
- A `fail_stop` channel admits only crashes of its sender.

All faults take effect before the **end of finite failures** (`EFF`). After `EFF`, no new fault starts, and the protocol has the rest of the run to converge.

## Lineage

Every run records the following:

- Every execution: an async run, or one execution of a tick.
- Every send, and what happened to it.
- Every receive, and which delivery it consumed.
- Which execution produced each output.

The **derivation** of the specification's result is the set of deliveries reached by walking backward from the executions that produced it. Hydrofrac can compute derivations at four levels of precision:

| Level | What a tick execution is taken to depend on |
|---|---|
| `Exec` | Every input of this execution and of every earlier execution of the same tick. |
| `Slice` (default) | The same, narrowed by a static slice of the program for each sink. |
| `Record` | Record-level provenance. An IR rewrite (`SimFlow::with_provenance`) threads a provenance set through every operator. |
| `Counterfactual` | Each node is re-executed alone, with one delivery removed at a time. A delivery is in the derivation if and only if the node stops producing the output without it. |

`Exec` and `Slice` always work, but they can over-approximate: they can include deliveries the output did not actually need. `Record` and `Counterfactual` are more precise, but they sometimes fail on a node:

- The provenance rewrite cannot see through every closure.
- A counterfactual replay that keeps every delivery sometimes fails to reproduce the node's output. In that case, removing deliveries one at a time proves nothing.

When a level fails on a node, hydrofrac uses the next less precise level for that node. `Counterfactual` falls back to `Record`, and `Record` falls back to `Slice`.

The fallback always moves toward over-approximation, and the direction matters:

- A derivation that is too large costs extra hypotheses, because the search tries fault sets it did not need to. The verdict is still correct.
- A derivation that is too small could leave out the delivery whose loss breaks the program, which would produce a false certificate.

Because every fallback moves toward the safe direction, certificates remain sound.

## The search loop

The search is Molly's loop, in Hydro terms:

1. The probe fixes `FACTS` and produces the first derivation.
2. Each derivation becomes a **clause**: a set of deliveries, at least one of which any breaking fault set must kill. An omission kills one message. `crash(node after m)` kills every send by that node after `m`.
3. A **hypothesis** is a fault set that hits every clause so far and stays within the crash budget.

4. Hydrofrac runs each hypothesis in the simulator, which explores schedules under it. There are two possible results: - The specification fails in some schedule. That schedule is the counterexample. - The specification holds. The run's derivation becomes a new clause.
5. The search stops when one of three things happens: - No hypothesis exists within the bounds. This produces a certificate. - A counterexample is found. - The hypothesis budget runs out (`Outcome::Budget`).

Hydrofrac chooses fault sets, and the simulator chooses interleavings under each one. The simulator compiles the flow into a single-process executable. In that executable, every channel is a queue, every tick is an explicitly scheduled batch, and every input and timer is driven by the test body.

### A worked example

Consider `broadcast_closed` from a process to a three-member cluster over `fail_stop`, with inputs `1` and `2`:

1. The probe delivers both items to every member, so `FACTS = {1, 2}`.
2. The probe's derivation names the source's sends to each member.
3. Each member's copy of item `2` exists only because the source sent it directly. One crash that falls after the send to member 1 and before the sends to the others leaves the survivors in disagreement.
4. Hydrofrac reaches that crash on its fifth hypothesis: `crash(source after bcast[→1]:Item(2))`.

Now consider echo reliable broadcast under the same bounds:

1. The same crash is tried.
2. Member 1 re-broadcasts what it received, so the other members still get item `2`.
3. The specification holds, and the run's derivation, which goes through member 1's echo, becomes a new clause. To break the protocol, a fault set must now kill both the source's send and the echo.
4. With one crash allowed and no omissions available on `fail_stop`, no single fault hits every clause.
5. The search runs out of hypotheses after 10 and reports a certificate.

## What a verdict means

A certificate holds under four conditions:

- **It assumes the program's declared fault model.** It covers omissions on lossy channels, and crashes of the nodes the test registered as crashable.
- **It assumes the test's bounds.** These are the crash budget, `EFF`, and the horizon. The horizon is however long the test body drives the run.
- **It covers one execution.** That execution is the cluster sizes, inputs, and timer firings the test fixes.
- **It covers only the schedules explored.**
- Under `Exploration::Exhaustive`, every batching and ordering is explored for every hypothesis, and the certificate is a proof within the bounds.
- Under `Exploration::Fuzz { instances }`, a sample of schedules is explored, and the result is reported as a fuzz result rather than a certificate.

A certificate is a statement about one execution. Confidence that it generalizes to other inputs and larger clusters is the confidence any test gives you. That confidence is stronger here than for an ordinary test, because the search over faults is exhaustive within the bounds rather than random.

A counterexample is a statement about the program: there exists a run in which the label is false.

### Why certificates from the generated specification are sound

The argument for completeness is the standard LDFI argument. If a fault set leaves every delivery in the specification's derivation intact, it cannot falsify the specification. That argument holds only for properties that are monotone in deliveries, meaning that extra deliveries can never turn a true result into a false one. "Every correct member holds every fact of a fixed set" is monotone by construction.

This is why `FACTS` is fixed by the probe rather than recomputed in each run. Consider the alternative property, "every member holds everything any member holds", evaluated over whatever each run happens to produce. That property is not monotone. A fault could cause one member to derive a new fact, which would falsify the property without touching any recorded derivation. Grading such a property would require a syntactic analysis of the specification, which hydrofrac does not have. The generated specification does not need one.

Hand-written specifications, described below, carry no such argument. Their certificates print as **RELATIVE**: valid within the fault model, relative to the form of the specification.

### What a verdict does not claim

A verdict makes no claim about agreement, liveness, or other inputs and topologies.

- **Agreement.** That members hold the same contents is the type system's conclusion, given the premise. Hydrofrac checks the premise and does not re-prove the conclusion.
- **Liveness.** That every member eventually holds every fact is not checked. The specification is evaluated at the end of a finite run.
- A protocol that would have converged with more time is reported as violated. That says as much about the test body as about the protocol.
- The counterexample shows what was still missing when the run ended.
- **Other inputs and topologies.** A verdict covers only the execution the test fixes.

## Sorting existing Hydro code

With one crash allowed and every registered node crashable, I ran hydrofrac under the generated specification against two sets of programs:

- Every producer of the `EventualConsistency` label in the codebase.
- Every demonstration protocol in `hydro_std::ec_inference_demos` whose labelled output is a set held at a cluster.

The tests are `hydro_test/src/cluster/ec_ldfi.rs` and `ec_demos_ldfi.rs`. Hypothesis counts include the probe run.

### Violated

These programs carry the label but do not have complete EC.

| Program | Hypotheses | Counterexample |
|---|---|---|
| `broadcast_closed`, process → cluster, `fail_stop` | 5 | `crash(source after bcast[→1]:Item(2))`. The sender dies between two sends of one item. |
| `broadcast_closed`, process → cluster, `lossy_delayed_forever` | 2 | `omit(bcast[→0]:Item(2))`. One message is lost and nothing retransmits it. |
| `broadcast_closed`, cluster → cluster, `fail_stop` | 5 | `crash(source@0 after bcast[0→1]:Item(10))` |
| `broadcast_live_from_process` (one hop over live membership) | 5 | `crash(source after bcast[→1]:2)` |
| `leader_merge_broadcast` | 2 | `crash(leader after merged[→1]:(writer 0, 100))`. The demo's documentation says its label assumes the leader survives. This is that assumption turned into a counterexample. |

## Certified

These programs have complete EC within the bounds.

| Program | Hypotheses | Exploration |
|---|---|---|
| Echo reliable broadcast (`hydro_test` version) | 10 | exhaustive |
| `reliable_broadcast_closed`, `reliable_broadcast_live` | 10 each | exhaustive |
| `uniform_reliable_broadcast_closed`, thresholds 1 and 2 | 28 each | 8 sampled schedules per hypothesis |
| `g_set_gossip`, `g_set_gossip_live` | 13 each | 8 sampled schedules |
| `multi_paxos`, the learners' `learned` output (1 proposer, 3 acceptors, 3 learners) | 24 | 8 sampled schedules |

Three of these rows are fuzz results, not proofs: uniform broadcast, gossip, and multi-Paxos have schedule spaces too large to explore exhaustively.

In `multi_paxos`, the learners re-broadcast what they learn, so the certificate is really a certificate for that echo cycle. The proposer is held correct.

## The pattern

Every single-hop dissemination fails under one crash or one lost message. Every protocol with an echo or gossip cycle holds. The label is a promise about the program that uses the transport, and only a program that re-disseminates keeps it.

## Not in scope

The remaining labelled collections are not questions about faults:

- `source_interval` on a cluster produces a stream of `()`.
- Keyed folds with algebraic proofs preserve the label given the same inputs.
- `collect_quorum` is a function of the response set.

The remaining demos are out of scope for different reasons:

- `quorum` and `epoch_splice` are combinators with no transport of their own. They take EC in and give EC out, so their premise belongs to whichever broadcast feeds them.
- `synod` labels nothing `EventualConsistency`.
- `abd`'s replica register is in scope in principle. However, it is a lattice `Singleton`, which the set-based specification cannot observe correctly. See the limitations below.

## Which collections a test checks

Today a test checks one labelled collection: the one it passes to `spec.observe_cluster`. A program may label many collections (multi-Paxos labels thirteen), and a test says nothing about the ones it does not name.

To make the scope of each certificate visible, every search begins by printing a list of every place in the program where a collection acquires the label. Each entry is marked with one of the following:

- `checked by this test`
- `UNCHECKED (fed by a network channel)`
- `nothing to check (local inputs only)`

**In progress: automatic coverage.** The specification depends only on the collection, and the labelled collections are visible in the program's IR. Checking them therefore does not need the test author. The next version will do the following:

1. Observe every labelled cluster collection that is fed by the network. This is done by an IR rewrite at flow finalization, in the same way the provenance rewrite already inserts operators.
2. Run one search per collection against the shared test body.
3. Report a verdict for each collection: - certified; - violated; - not in scope, with the reason, such as a lattice `Singleton` or a collection inside a tick.

The printed list then goes away. `spec.observe_cluster` remains for checking a single collection by hand.

## Hand-written specifications

A `Spec` can also be written by hand, as two Hydro views over the program's outputs:

- A **pre-condition**. If it is empty, the run is ignored.
- A **post-condition**. If it is empty while the pre-condition is not, the run is a counterexample.

`Ldfi` also accepts simpler goals over plain outputs: `goal`, `goal_all_members`, and `goal_witnessed`.

Hand-written specifications let hydrofrac check properties the type system does not state. The current tests cover the following:

| Property | Test file |
|---|---|
| Durability of committed entries in Raft | `raft_ldfi.rs` |
| Durability of committed entries in Paxos | `paxos_ldfi.rs` |

| Property | Test file |
|---|---|
| Atomicity in two-phase commit | `two_pc_ldfi.rs` |
| Agreement in ISR replication | `isr_replication.rs` |
| Claim counting in quorum | `quorum_ldfi.rs` |
| Retry-until-ack delivery | `reliable_delivery.rs` |

Certificates from these tests are graded RELATIVE, and hydrofrac says so.

### Bugs found

These tests have found real bugs.

- **Raft.** An injected bug in Raft's election restriction compared logs by index only. Hydrofrac found it in 90 hypotheses, using four omissions on two links and no crash. That scenario was not one the author had anticipated. Correct Raft certifies in 235 hypotheses under the same bounds. Both tests are `#[ignore]`d because they take thirty minutes and an hour.
- **Paxos.** The P1b quorum counted replies rather than distinct acceptors. This is a defect in the production protocol. It became visible once the body's `EFF` moved past the first ballot. The following now certify, and the three tests together take 45 minutes and are `#[ignore]`d:

| Protocol | Hypotheses |
|---|---|
| Correct Paxos | 372 |
| The Paxos key-value store | 301 |
| Compartmentalized Paxos | 568 |

The simulator also found two defects that are not about faults:

- `collect_quorum_with_response` was not a function of its input, so its `nondet!` justification was false. It is fixed, and a test covers it.
- A `fold` with an idempotence proof had no simulator implementation of its retry-only cast. The cast is now the identity, since the simulated network never duplicates messages.

## Limitations

- **One execution.** A certificate covers the test's topology and inputs. Generalizing it is the test author's job.
- **Sampled schedules are not certificates.** Protocols with large schedule spaces run under `Fuzz`, and their results are reported that way.
- **The test body sets the horizon.** A protocol that has not converged by the end of the run is reported as a counterexample, not as vacuous.
- A convergence witness reported by the protocol would fix this. Such a witness would say that every correct member has settled on the same view. It is not built.
- Raft's hand-written pre-condition ("some member committed") shows the cost: its test body was tuned until correct Raft converged.
- **Set-valued collections only.** The generated specification observes a stream of facts. A lattice `Singleton`, such as `abd`'s max-by-timestamp register, needs a variant that observes each member's final value and checks the values for equality. That variant is not built.

- **Static membership.** In the simulator, every member is joined from the start. The `_live` demos are tested for faults, not for join timing.
- **Lineage gaps.**
- The provenance rewrite does not handle `entries_partially_ordered` on a keyed stream, so `leader_merge` runs at `Level::Slice`.
- Counterfactual replay can panic inside a program that assumes a delivery it is being denied. The panic is contained on the generated side of the dynamic-library boundary, and the result for that node is treated as inconclusive.
- **The list of unchecked labels does not fail a test.** Making it fail a test requires a notion of "every search over this program", which a per-program registry of specifications could provide.
- **Some fault types are not modelled.**
- Byzantine faults are not modelled at all.
- Partitions can be expressed as sets of omissions on lossy channels, and are found that way. The Raft counterexample is four omissions on two links.
- Duplication has no fault point.

## Next steps

These are ordered by how much they contribute to the goal of checking every label automatically:

1. Automatic coverage of every labelled collection (in progress).
2. A lattice-aware specification for `Singleton` outputs, and `abd` checked under it.
3. A convergence witness in the generated specification for protocols that can report one. The horizon then becomes a parameter rather than a source of false verdicts.
4. Decision-trace replay: record every nondeterministic decision in a run, then replay a prefix with one decision changed. This is the general form of counterfactual lineage, of which node-level replay is a coarse special case.
5. Raft's `committed` collection checked under the generated specification. The test harness is written; the run takes an hour.